MODULE 14

Production Patterns & Technical Debt

Duration	Format	Audience	Day / Position
45 min	**Lecture + Demo**	**Mixed Engineers**	**Day 2 / Sixth**

LEARNING OBJECTIVES

By the end of this module, participants will be able to:

1. Apply structured error handling patterns to AI-generated code using Claude Code prompts that enforce try/except, logging, and graceful degradation
2. Use Claude Code for performance analysis: identify bottlenecks, optimize hot paths, and benchmark before/after improvements
3. Measure code quality metrics (complexity, maintainability, duplication) and use Claude to refactor toward team thresholds
4. Manage dependencies safely with Claude Code: audit outdated packages, detect vulnerabilities, and generate upgrade plans
5. Conduct systematic technical debt analysis with Claude: identify, classify, prioritize, and create remediation plans for tech debt
6. Integrate production readiness checks into the CI/CD pipeline built in Module 9

SESSION TIMING BREAKDOWN

0:00 – 0:09	**Section 1: Error Handling Patterns for AI-Generated Code**
0:09 – 0:18	**Section 2: Performance Optimization with Claude Code**
0:18 – 0:27	**Section 3: Code Quality Metrics & Refactoring**
0:27 – 0:36	**Section 4: Dependency Management & Security**
0:36 – 0:43	**Section 5: Technical Debt Analysis**
0:43 – 0:45	**Wrap-Up: Quick Reference & Transition to Lab 3**

0:00 – 0:09	SECTION 1: Error Handling Patterns for AI-Generated Code

1. Error Handling Patterns for AI-Generated Code

1.1 The AI Error Handling Problem

AI-generated code often has a blind spot: error handling. Claude generates correct happy-path logic but may skip edge cases, swallow exceptions, or produce generic try/except blocks. Production code needs structured, specific, and informative error handling.

AI Default (Weak)	Production Pattern (Strong)
try: ... except Exception: pass	try: ... except ValueError as e: logger.error(f"Validation failed: {e}"); raise
No logging	Structured logging with context (sensor_id, timestamp, values)
Generic error messages	Specific messages: "Temperature {value} out of range [-40, 150] for sensor {id}"
Crash on bad input	Graceful degradation: skip bad record, log it, continue pipeline
No retry logic	Retry with backoff for transient failures (network, API timeouts)

1.2 Prompting for Production Error Handling

```
# Prompt Claude for production-grade error handling:
> Refactor validate_reading() to add production error handling:
> 1. Catch specific exceptions (ValueError, TypeError, KeyError)
> 2. Add structured logging with sensor_id context
> 3. Return a ValidationResult with errors list instead of
>    raising exceptions (fail-soft for batch processing)
> 4. Add @functools.wraps for decorated functions
> 5. Include a timeout for any I/O operations

# Result: Claude generates something like:
def validate_reading(reading: SensorReading) -> ValidationResult:
    errors: list[str] = []
    try:
        if not (-40 <= reading.temperature <= 150):
            errors.append(
                f"Temperature {reading.temperature} out of range "
                f"[-40, 150] for sensor {reading.sensor_id}"
            )
    except (TypeError, AttributeError) as e:
        logger.error(
            "Validation error for sensor %s: %s",
            reading.sensor_id, e,
            extra={"sensor_id": reading.sensor_id}
        )
        errors.append(f"Unexpected error: {e}")
    return ValidationResult(valid=len(errors) == 0, errors=errors)
```

> ☆ **KEY POINT:** The key shift: instead of “write a validator”, prompt “write a production validator with structured logging, specific exception handling, and fail-soft behavior for batch processing.” Adding “production” to your prompts triggers more robust patterns.

0:09 – 0:18	SECTION 2: Performance Optimization with Claude Code

2. Performance Optimization with Claude Code

2.1 Identify Bottlenecks

```
# Ask Claude to profile and identify bottlenecks:
> Analyze the performance of our sensor toolkit pipeline.
> Identify:
> 1. Functions with O(n²) or worse complexity
> 2. Unnecessary memory copies (list comprehensions that
>    should be generators)
> 3. Repeated I/O operations that could be batched
> 4. Functions called in tight loops that could be cached
> For each issue, estimate impact and suggest a fix.

# Ask Claude to benchmark:
> Add timing benchmarks to the pipeline:
> - Time each stage: read → validate → clean → analyze
> - Use time.perf_counter() for accurate measurement
> - Output: stage name, duration, records processed, rate
```

2.2 Common Optimizations

Pattern	Before	After
Generator vs. list	cleaned = [clean(r) for r in readings]	cleaned = (clean(r) for r in readings) # lazy
Batch I/O	for r in readings: write_row(r)	writer.writerows(readings) # single call
Cache expensive calls	@lru_cache on pure functions	from functools import lru_cache
Vectorize	for i in range(len(data)): data[i] *= scale	data = [x * scale for x in data] # or numpy
Early exit	validate all fields then return	if not valid_id: return early # skip remaining checks
Pre-compile regex	re.match(pattern, s) in loop	compiled = re.compile(pattern); compiled.match(s)

```
# Ask Claude to optimize a specific function:
> Optimize process_batch() for 10,000+ readings.
> Current: 4.2 seconds for 10K readings.
> Target: under 1 second.
> Profile first, then apply the most impactful optimization.
> Show before/after benchmarks.
```

☆ **KEY POINT:** Always benchmark before and after optimization. Claude can suggest optimizations that look clever but don't actually improve performance (or make it worse). The prompt pattern: "Current: X seconds. Target: Y seconds. Profile first, then optimize."

Track A (IT Engineers)	Track B (Process Engineers)
Optimize API response times: connection pooling, async handlers, query optimization	Optimize data pipeline throughput: batch I/O, generators, memory-efficient CSV processing
Profile with cProfile/py-spy, optimize hot paths	Profile with time.perf_counter(), optimize largest bottleneck

0:18 – 0:27	SECTION 3: Code Quality Metrics & Refactoring

3. Code Quality Metrics & Refactoring

3.1 Measuring Quality

```
# Ask Claude to measure code quality:
> Analyze code quality metrics for src/sensor_toolkit/:
> 1. Cyclomatic complexity (radon) - flag functions > C
> 2. Maintainability index (radon mi) - flag files < A
> 3. Code duplication - find repeated patterns
> 4. Function length - flag functions > 30 lines
> 5. Import complexity - count import depth
> Create a quality report with scores and recommendations.

# Install measurement tools:
> !pip install radon
> !radon cc src/ -a -nc  # Complexity > C only
> !radon mi src/ -nc     # Maintainability < A only
```

Metric	Tool	TI Threshold
Cyclomatic complexity	radon cc	Max C (no D or F functions)
Maintainability index	radon mi	Min A (no B or C files)
Line count per function	wc -l / Claude analysis	Max 30 lines (prefer 15-20)
Duplication	Claude analysis / jscpd	Max 5% duplicated code
Test coverage	pytest --cov	Min 80% (validators: 100%)
Type hint coverage	mypy --strict	100% for public functions

3.2 AI-Assisted Refactoring

```
# Ask Claude to refactor based on metrics:
> The function clean_batch() has cyclomatic complexity F
> (score 22). Refactor to reduce complexity below C (10):
> - Extract helper functions for each cleaning step
> - Replace nested if/else with early returns
> - Use dictionary dispatch for sensor-type-specific logic
> - Maintain all existing tests passing
> Show the refactored code and new complexity score.
```

☆ **KEY POINT:** Quality metrics give Claude a concrete target. Instead of “refactor this,” say “reduce complexity from F(22) to C(10) using extract-function and early-return patterns.” Claude produces better refactors with specific numeric goals.

0:27 – 0:36	SECTION 4: Dependency Management & Security

4. Dependency Management & Security

4.1 Dependency Audit

```
# Ask Claude to audit dependencies:
> Audit our project dependencies in requirements.txt:
> 1. List all packages with current and latest versions
> 2. Flag any packages with known security vulnerabilities
> 3. Identify packages that are 2+ major versions behind
> 4. Check for unused packages (installed but not imported)
> 5. Generate an upgrade plan with risk assessment
# Use pip-audit for vulnerability scanning:
> !pip install pip-audit
> !pip-audit
# Ask Claude to fix vulnerabilities:
> pip-audit found 2 vulnerabilities:
> [paste pip-audit output]
> Create an upgrade plan that fixes these without
> breaking our existing tests.
```

4.2 Dependency Pinning Strategy

File	Purpose
requirements.txt	Production dependencies, pinned exact versions: pandas==2.1.4
requirements-dev.txt	Dev-only tools (pytest, ruff, black), pinned exact
pyproject.toml	Project metadata, dependency ranges for libraries
.github/dependabot.yml	Automated dependency update PRs from GitHub

```
# Generate a Dependabot config with Claude:
> Create .github/dependabot.yml that:
> - Checks pip dependencies weekly
> - Groups minor/patch updates into one PR
> - Keeps major updates as separate PRs
> - Assigns reviews to the team lead
# Claude generates:
version: 2
updates:
  - package-ecosystem: pip
    directory: /
    schedule:
      interval: weekly
    groups:
      minor-and-patch:
        update-types: [minor, patch]
```

⚠ **WARNING:** Never run pip install on a production system without testing first. Always test dependency upgrades in CI before deploying. Claude can suggest upgrades, but you must verify they don't break existing functionality by running the full test suite.

0:36 – 0:43	SECTION 5: Technical Debt Analysis

5. Technical Debt Analysis

5.1 Systematic Debt Identification

```
# Ask Claude for a comprehensive tech debt audit:
> Perform a technical debt analysis of this project:
> 1. IDENTIFY: Find all TODO/FIXME/HACK comments
> 2. CLASSIFY: Group by type (design debt, code debt,
>    test debt, documentation debt, dependency debt)
> 3. PRIORITIZE: Rate each by impact (High/Medium/Low)
>    and effort to fix (Easy/Medium/Hard)
> 4. PLAN: Create a prioritized remediation plan
>    (high-impact + easy-effort items first)
> Output as a Markdown table in docs/tech-debt.md
```

Debt Type	Examples	Claude Prompt Pattern
Design debt	God functions, tight coupling, missing abstractions	> Identify functions with more than 3 responsibilities
Code debt	Duplication, dead code, magic numbers, poor naming	> Find repeated code patterns and magic numbers
Test debt	Missing tests, low coverage, no edge case tests	> Find public functions without test coverage
Documentation debt	Missing docstrings, outdated README, no API docs	> Find public functions without docstrings
Dependency debt	Outdated packages, unused imports, security vulns	> Audit dependencies for outdated/vulnerable packages

5.2 The Debt Quadrant

High Impact + Easy Fix (🚀 Do First)	High Impact + Hard Fix (📅 Plan)
Missing input validation on public API	Replace custom CSV parser with pandas
Hardcoded sensor ranges → move to config	Redesign pipeline for async processing
Add missing docstrings to validators	Extract common patterns into shared library

Low Impact + Easy Fix (✅ Quick Wins)	Low Impact + Hard Fix (❌ Defer)
Remove dead code and unused imports	Migrate from unittest to pytest (tests already pass)
Fix TODO comments in utility functions	Complete type hint coverage for internal modules

Update README with current setup instructions	Refactor logging to use structured JSON format

☆ **KEY POINT:** Run the tech debt audit monthly. Add it to your CI pipeline: Claude scans for new TODOs, measures complexity trends, and flags regressions. The goal isn't zero debt — it's managed debt with a clear remediation plan.

0:43 – 0:45	WRAP-UP: Quick Reference & Transition to Lab 3

6. Quick Reference & Transition to Lab 3

MODULE 14 RECAP

✅ Error handling: prompt for “production” patterns — specific exceptions, structured logging, fail-soft behavior, ValidationResult objects.

✅ Performance: profile first (time.perf_counter), set numeric targets, benchmark before/after. Generators, batch I/O, caching.

✅ Code quality: radon for complexity/maintainability, Claude refactors toward numeric thresholds (complexity C → A).

✅ Dependencies: pip-audit for vulnerabilities, Dependabot for updates, pin exact versions, test upgrades in CI first.

✅ Tech debt: IDENTIFY → CLASSIFY → PRIORITIZE → PLAN. Use the debt quadrant (high impact + easy fix first).

✅ Integration: add quality gates (radon, pip-audit, coverage) to CI pipeline from Module 9.

QUICK REFERENCE CARD

Task	Prompt Pattern	Tool
Error handling	> Refactor with production error handling: specific exceptions, logging, fail-soft	Claude Code
Profile performance	> Analyze performance, identify $O(n^2)$, benchmark each pipeline stage	time.perf_counter
Optimize	> Current: Xs, Target: Ys. Profile first, then optimize.	Claude + benchmarks
Measure complexity	> !radon cc src/ -a -nc	radon
Refactor to target	> Reduce complexity from F(22) to C(10) using extract-function	Claude + radon
Audit dependencies	> !pip-audit	pip-audit
Generate Dependabot	> Create .github/dependabot.yml for weekly pip checks	Claude Code
Tech debt audit	> Perform tech debt analysis: IDENTIFY, CLASSIFY, PRIORITIZE, PLAN	Claude Code

TRANSITION TO LAB 3

Lab 3 (Module 15) is the Full Feature Sprint — 60 minutes of end-to-end feature development using every technique from the course. You’ll decompose a feature, implement with subagents, test with quality gates, refactor with metrics, and deliver a PR-ready result. This is where everything comes together.